

XHAND Python SDK接口说明文档

版本记录

版本	日期	修改记录	操作人
1.2.10	2024.11.28	初版	孙焜
1.2.11	2024.12.09	新增低阻尼功能使用说明，以及中英文 README.md	孙焜

✨ 示例代码详见 xhand_control_example.py

X-hand Python API 参考

XHandControl

XHandControl是灵巧手sdk主要API接口类，与灵巧手的通信均需要通过调用该类的相应方法来实现，以下是XHandControl 类的成员函数。

enumerate_devices | 枚举设备

接口功能

枚举设备，可选择单独一个设备进行通信

接口声明

```
Python
def enumerate_devices(comm_type: str) -> list[str]
```

注意事项

无

参数说明

参数名称	类型	是否必选	示例值	说明
comm_type	str	是	"RS485" 或 "EtherCAT"	枚举当前可用的设备名

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	list[str]		返回串口设备名或者网卡名

打开设备

open_serial | 打开串口设备

接口功能

根据指定的端口和波特率尝试连接X-hand设备

接口声明

```
Python
def open_serial(port: str, baudrate: int) -> ErrorStruct
```

注意事项

连接成功后才能与X-hand进行正常通信

当前串口使用的波特率为3000000

参数说明

该接口的参数说明如下：

参数名称	类型	是否必选	示例值	说明
port	str	是	"/dev/ttyUSB0"	
baudrate	int	是	3000000	通信串口使用的波特率

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	ErrorStruct	无	error_code为0代表成功，否则代表失败，参考 ErrorStruct 错误码类

open_ethercat | 打开EtherCAT设备

接口功能

打开 EtherCAT 设备

接口声明

```
Python
def open_ethercat(ifname: str) -> ErrorStruct
```

注意事项

连接成功后才能与X-hand进行正常通信

参数说明

该接口的参数说明如下：

参数名称	类型	是否必选	示例值	说明
ifname	str	是	enps0	

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	ErrorStruct	无	error_code为0代表成功，否则代表失败，参考 ErrorStruct 错误码类

list_hands_id | 获取所有手ID

接口功能

获取所有已连接的手ID

接口声明

```
Python
def list_hands_id() -> list[int]
```

注意事项

必须先调用打开设备接口成功后，调用该接口才能返回连接的手ID

参数说明

无

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	list[int]		所有已连接的手ID

set_hand_id | 设置手ID

接口功能

设置手的ID

接口声明

```
Python
def set_hand_id(old_id: int, new_id: int) -> ErrorStruct
```

注意事项

手ID值范围为 0 ~ 125

参数说明

参数名称	类型	是否必选	示例值	说明
old_id	str	是	0	0~125
new_id	int	是	1	0~125

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	ErrorStruct	无	error_code为0代表成功，否则代表失败，参考 ErrorStruct 错误码类

close_device | 关闭设备

接口功能

关闭当前打开的X-hand设备

接口声明

```
Python
def close_device() -> None
```

注意事项

退出程序前需关闭X-hand设备，保证设备正常退出

参数说明

无

返回值

无

read_device_info | 读取设备信息

接口功能

读取打开的X-hand设备信息

接口声明

```
Python
def read_device_info(device_id:int) -> dict[ErrorStruct, DeviceInfo_t]
```

注意事项

如果未读取到信息，所有信息数据都为0

参数说明

参数名称	类型	是否必选	示例值	说明
device_id	int	是	0	整手ID

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	dict[ErrorStruct, DeviceInfo_t]		error_code为0代表成功，否则代表失败，参考 ErrorStruct 错误码类 。 参考 DeviceInfo_t 设备信息类

read_version | 读取版本信息

接口功能

读取X-hand设备的版本信息

接口声明

```
Python
def read_version(device_id: int, component_id: int) -> dict[ErrorStruct, str]
```

注意事项

如果未读取到信息，返回空字符串

参数说明

参数名称	类型	是否必选	示例值	说明
component_id	int	是	0	详见 通信ID
device_id	int	是	0	整手ID

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	dict[ErrorStruct, str]		error_code为0代表成功，否则代表失败，参考 ErrorStruct 错误码类

send_command | 发送控制指令

接口功能

下发关节指令，最高支持 83 Hz

接口声明

```
Python
def send_command(device_id: int, command: HandCommand_t) -> ErrorStruct
```

注意事项

res为预留参数，无需赋值

参数说明

该接口的参数说明如下：

参数名称	类型	是否必选	示例值	说明
device_id	int	是	0	整手ID
command	HandCommand_t	是		HandCommand_t 整手运动控制类

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	ErrorStruct		error_code为0代表成功，否则代表失败，参考 ErrorStruct 错误码类

read_state | 读取状态

接口功能

读取当前x-hand状态信息

接口声明

```
Python
def read_state(device_id: int, force_update = false) -> dict[ErrorStruct, HandState_t]
```

注意事项

如果为串口通信，下发控制指令后会自动刷新 `HandState_t`，如果需要强制刷新 `HandState_t`，需要一定耗时

参数说明

参数名称	类型	是否必选	示例值	说明
device_id	int	是	0	整手ID
force_update	bool	否	false	如果为true,会发送状态刷新指令，会有一定阻塞时间

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	dict[ErrorStruct, HandState_t]		error_code为0代表成功，否则代表失败，参考 ErrorStruct 错误码类 ，参考 整手状态类

~~calibrate_joint | 关节标定 (旧版标定，之后逐步废弃)~~

接口功能

标定X-hand关节控制参数

接口声明

```
Python
def calibrate_joint(device_id: int, step: int, angles_limit = {}) -> ErrorStruct
```


注意事项

标定X-hand所有关节的控制参数，标定参数异常可能导致关节运动异常，请谨慎使用

参数说明

该接口的参数说明如下：

参数名称	类型	是否必选	示例值	说明
device_id	int	是	0	整手ID
step	int	是	[1,5]	标定分为5个阶段，需要按照顺序执行
angles_limit	list[int]	否	[0, 90, -70, 80, 0, 90, 12, -8, 0, 110, 0, 110, 0, 110, 0, 110, 0, 110, 0, 110, 0, 110]	关节运动的限位角度，不同型号可能有不同值，详细信息，请咨询技术人员

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	ErrorStruct	无	error_code为0代表成功，否则代表失败，参考 ErrorStruct 错误码类

calibrate_joint_by_mold | 关节标定（模具）

接口功能

借助模具自动标定X-hand关节控制参数

接口声明

```
Pythonhon
def calibrate_joint_by_mold(device_id: int, tools: str, step: int, calibrated_angles = [], angles_limit= []) -> ErrorStruct
```

注意事项

标定X-hand所有关节的控制参数，标定参数异常可能导致关节运动异常，请谨慎使用；

标定过程中尽量保证X-hand紧贴模具

参数说明

该接口的参数说明如下：

参数名称	类型	是否必选	取值	说明
device_id	int	是	0	整手ID
tools	str	是	"S1", "D1"	标定模具型号
step	int	是	Int, 取值为 1 或 2	单模具只需要1步，多模具由模具具体个数决定
calibrated_angles	list[int]	否	- 如果tools 取 S1, 则只需一步, 则 step=1; calibrated_angles = [10, 60, 30, 0, 10, 10, 10, 10, 10, 10, 10, 10] - 如果 tools 取 "D1", 则分两步, step=1, calibrated_angles=[10, 30, 60, 0, 10, 10, 10, 10, 10, 10, 10, 10]; step=2, calibrated_angles=[50, 60, 45, 10, 30, 30, 30, 30, 30, 30, 30, 30]	标定角度，根据模具的固定角度进行确定
angles_limit	list[int]	否	M2.25: [0, 105, -60, 90, -10, 105, -10, 10, 0, 110, 0, 110, 0, 110, 0, 110, 0, 110, 0, 110, 0, 110, 0, 110];	关节运动的限位角度，不同型号可能有不同值，详细信息，请咨询技术人员

调用方式

- 单模具标定：调用一次 calibrate_joint_by_mold，step值为1，传递一次 calibrated_angles
- 多模具标定：需要调用两次 calibrate_joint_by_mold，step值分别为1和2，传递两次 calibrated_angles，具体值如上表所示

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
------	----	-----	----

返回值	ErrorStruct	无	error_code为0代表成功，否则代表失败，参考 ErrorStruct 错误码类
-----	-------------	---	---

reset_sensor | 传感器复位

- 接口功能
 - 复位指尖压力传感器
- 接口声明

```
Python
def reset_sensor(device_id:int, sensor_id: int) -> ErrorStruct
```

- 注意事项
 - 部分使用情况下，需要先复位传感器，保证传感器数值的准确性
- 参数说明

参数名称	类型	是否必选	示例值	说明
device_id	int	是		整手ID
sensor_id	int	是	[17,21]	Sensor id从17开始，从大拇指依次增加

- 返回值
 - 该接口的返回值说明如下：

参数名称	类型	是否必选	示例值	说明
返回值	ErrorStruct	是		error_code为0代表成功，否则代表失败，参考 ErrorStruct 错误码类

get_hand_type | 读取手类型

- 接口功能

读取手的类型(L/R)

接口声明

```
Python
def get_hand_type(device_id: int) -> dict[ErrorStruct, str]
```

注意事项

无

参数说明

参数名称	类型	是否必选	示例值	说明
device_id	int	是	0	整手ID

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	dict[ErrorStruct, str]		error_code为0代表成功，否则代表失败，参考 ErrorStruct 错误码类

get_serial_number | 读取手序列号

接口功能

读取手的序列号

接口声明

```
Python
def get_serial_number(device_id: int) -> dict[ErrorStruct, str]
```

注意事项

无

参数说明

参数名称	类型	是否必选	示例值	说明
device_id	int	是	0	整手ID

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	dict[ErrorStruct, str]		error_code为0代表成功，否则代表失败，参考 ErrorStruct 错误码类

get_hand_name| 读取手名字

接口功能

读取手的名字

接口声明

```
Python
def get_hand_name(device_id: int) -> dict[ErrorStruct, str]
```

注意事项

无

参数说明

参数名称	类型	是否必选	示例值	说明
device_id	int	是	0	整手ID

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	dict[ErrorStruct, str]		error_code为0代表成功，否则代表失败，参考 ErrorStruct 错误码类

set_hand_name | 设置手名字

接口功能

设置手的名字

接口声明

```
Python
def set_hand_name(device_id: int, name: str) -> ErrorStruct
```

注意事项

名字不支持中文，最长不超过30个字符

参数说明

参数名称	类型	是否必选	示例值	说明
device_id	int	是	0	整手ID
name	str	是	"test_hand"	

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	ErrorStruct		error_code为0代表成功，否则代表失败，参考 ErrorStruct 错误码类

get_sdk_version | 获取 SDK 版本号

接口功能

获取 SDK 的 Python 版本号

接口声明

```
Python
def get_sdk_version() -> str
```

参数说明

无

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	str	1.1.1	

基础属性类

该章节介绍 X-hand Python API 涉及的基础属性类。

FingerCommand_t | 手指运动控制类

该类记录手指运动控制信息。类声明如下所示。

Python

```
class FingerCommand_t:
    def __init__(self) -> None:
        self.id: int
        self.kp: int      # 比例增益参数
        self.ki: int      # 积分系数
        self.kd: int      # 微分增益参数
        self.position: float # 设定位置
        self.tor_max: int   # 力矩上限
        self.mode: int      # 模式
        # 以下为暂时保留项，无特殊含义
        self.res0: int
        self.res1: int
        self.res2: int
        self.res3: int
```

该类成员变量说明如下：

变量名	变量类型	变量说明	示例值	变量数值范围

id	int	手指 ID	0	0~65535
kp	int	比例增益参数	0	0~5000
ki	int	积分系数	0	-3000~3000
kd	int	微分增益参数	0	0~5000
position	float	设定位置	0	参考灵巧手关节参数
tor_max	int	力矩上限	0	0~400
mode	int	模式	0	0：无力模式 3：位控模式 5：力控模式 其中低阻尼模式 PID 参数值建议： kp = 120 ki = 1600 kd = 12000

HandCommand_t | 整手运动控制类

该类记录整手运动控制信息，由12组手指运动控制类（FingerCommand_t）构成。类声明如下所示。

```
Python
class HandCommand_t:
    def __init__(self) -> None:
        self.finger_command: list[FingerCommand_t]
```

HandState_t | 整手状态类

该类记录整手状态信息，包含有关节位置以及传感器状态。类结构声明如下所示。

```
Python

# PXS_R_ForceData 返回的原始数值需要除以 10 才能得到真实的力（单位：N）。
# 例如：
```



```
# fingertip calc_pressure = [59, 74, 255]
```

```
# 对应的实际力为：
```

```
# [5.9 N, 7.4 N, 25.5 N]
```

```
class PXSR_ForceData:
```

```
def __init__(self) -> None:
```

```
    self.fx: int
```

```
    self.fy: int
```

```
    self.fz: int
```

```
class SensorData_t:
```

```
def __init__(self) -> None:
```

```
    self.calc_force: PXSR_ForceData
```

```
    self.raw_force: list[PXSR_ForceData]
```

```
    self.temperature: list[int]
```

```
    self.calc_temperature: int
```

```
class FingerState_t:
```

```
def __init__(self) -> None:
```

```
    self.id: int
```

```
    self.sensor_id: int
```

```
    self.position: float # 解析后的位置值
```

```
    self.torque: int
```

```
    self.raw_position: int
```

```
    self.temperature: int # 高8bit存储掌板温度 低8bit存储关节板温度
```

```
    self.commboard_err: int # 掌板错误
```

```
    self.jointboard_err: int # 指关节错误
```

```
    self.tipboard_err: int # 指尖传感器错误
```

```
    self.default5: int
```

```
    self.default6: int
```

```
    self.default7: int
```

```
class HandState_t:
```

```
def __init__(self) -> None:
```

```
    self.finger_state: list[FingerState_t]
```

```
    self.senser_data: list[SensorData_t]
```

DeviceInfo_t | 设备信息类

该类记录 设备 相关信息。类结构声明如下所示。

Python

```
class HandParam_t:
```

```
    def __init__(self) -> None:
```

```
        self.position_stretching: list[int]
```

```
        self.position_closing: list[int]
```

```
        self.position_zero: list[int]
```

```
        self.position_init: list[int]
```

```
        self.angle_stretching: list[int]
```

```
        self.angle_closing: list[int]
```

```
class CalibrationParams:
```

```
    def __init__(self) -> None:
```

```
        self.tools: list[str]
```

```
        self.step: int
```

```
        self.calibrated_angles: list[int]    # 标定角度（单位：度）：从spec获取
```

```
        self.lower_angles: list[int]        # 上位机获取的下限角度：从spec获取
```

```
        self.upper_angles: list[int]        # 上位机获取的上限角度：从spec获取
```

```
class CalibrationUnion:
```

```
    def __init__(self) -> None:
```

```
        self.handparam: HandParam_t
```

```
        self.calibr_params: CalibrationParams
```

```
class DeviceInfo_t:
```

```
    def __init__(self) -> None:
```

```
        self.resverse: int
```

```
        self.ev_hand: str
```

```
        self.hand_id: int
```

```
        self.serial_number: list[str]
```

```
        self.name: list[str]
```

```
self.hand_param: HandParam_t # 校准参数

self.iap_flag: int # 升级 ID

self.is_calibrated: int
```

CalibrationErrorInfo | 校准错误信息类

本结构体记录 **CalibrationErrorInfo** 相关信息。类结构声明如下所示。

Python

```
class CalibrationErrorInfo:
    def __init__(self) -> None:
        self.calib_status: int # 校准状态信息, uint8_t
        self.joint_number: int # 错误时返回错误的关节number (0~11) 或者0xffff (不对应某一关节参数时)
        self.calibrated_angle: int # 错误值: 校准角度
        self.calibrated_adc_value: int # 错误值: 校准原始值
        self.intercept: int # 错误值: 校准插值
```

ErrorStruct | 错误码类

该类记录错误码相关信息。类结构声明如下所示。

Python

```
class ErrorStruct:
    def __init__(self) -> None:
        self.error_code: int
        self.error_message: str
```

通信ID

手指关节ID

灵巧手由一个掌板、12个指关节板、5个指尖传感器板组成。每一个板由唯一ID作为通信的唯一标识，且不重复。

其中指关节板和指尖传感器板的ID，不可更改，其值如下：

名称	ID	名称	ID	名称	ID	名称	ID	名称	ID
----	----	----	----	----	----	----	----	----	----

大拇指偏摆	0	食指偏摆	3	中指1关节	6	无名指1关节	8	小指1关节	10
大拇指1关节	1	食指1关节	4	中指2关节	7	无名指2关节	9	小指2关节	11
大拇指2关节	2	食指2关节	5						
大拇指传感器	0x11	食指传感器	0x12	中指传感器	0x13	无名指传感器	0x14	小指传感器	0x15

灵巧手ID

灵巧手ID 默认为0，用户可以将修改。范围0~125。

掌板ID 是灵巧手ID的最高位置1，这样可以和指关节ID和指尖传感器ID 区分开

例如：灵巧手ID设置为3（0x03），实际通讯使用0x83作为掌板ID